Assignement 1
Nonlinear Least Square Method

Mael Madec

March 19, 2024

Contents

1. Introduction 3
2. Theoretical Background: 4
   2.1. Circle Computation: 4
   2.2. Optimization Techniques: 4
3. Implementation: 5
   3.1. Algorithms 5
   3.2. Main Loop 5
   3.3. Parameters and Initial Conditions: 6
4. Results 7
   4.1. Simple Levenberg-Marquardt 7
      4.1.1. Iteration = 500 7
      4.1.2. Iteration = 1500 9
      4.1.3. Iteration = 5000 10
   4.2. Weighted Levenberg-Marquardt 11
      4.2.1. Initial problem 11
      4.2.2. Introducing weighting 13
      4.2.3. Results 13
5. Conclusion 15
Bibliography 16
Index of Figures 17

1. Introduction

This report delves into the application and analysis of the Levenberg-Marquardt gradient descent algorithm for nonlinear problems, specifically focusing on one involving circles.

The report is structured to first provide a background on the problem understanding and the optimization techniques employed. It then details the implementation of these methodologies, including the algorithms and main loop used in the MATLAB script. The results section presents the findings from applying these techniques under various conditions and iterations, highlighting the performance and convergence behavior of the algorithms.

Finally, the report concludes with observations and comments on the effectiveness of the methodologies used.

2. Theoretical Background:

2.1. Circle Computation:

The coordinates provided in the document and the lengths returned by the provided MATLAB functions act as circle characteristics. As such, to find the coordinates of the phones is essentially to find the common point between the three circles. Given :

$$r_i^2 = (x - x_i)^2 + (y - y_i)^2 \tag{1}$$

The problem becomes :

$$r_1^2 = r_2^2 = r_3^3 \tag{2}$$

The coordinates (x, y) being the common point between the 3 radii, they act as the parameters on which the optimization techniques will be used for.

2.2. Optimization Techniques:

Two methods are used for optimization :

- Levenberg-Marquardt as the base method. The equation used is the following :

$$\theta_k = \theta_{k-1} - \eta_{k-1}\left(J_{k-1}^T J_{k-1} + \alpha_{k-1} I\right)^{-1} J_k^T e_{k-1} \tag{3}$$

  The error is determined by substracting the calculated radii (with parameters) with the provided radii. The damping is a fixed parameter.

- Weighted Levenberg-Marquardt for the study of the consideration of variance into the problem. With the introduction of a weighting matrix, the formula slightly changes to :

$$\theta_k = \theta_{k-1} - \eta_{k-1}\left(J_{k-1}^T W J_{k-1} + \alpha_{k-1} I\right)^{-1} J_k^T W e_{k-1} \tag{4}$$

  W is not a fixed weights matrix. Instead, it is updated by the ADAM algorithm, chosen for its ability to adjust the learning rate per each parameter, combined with its bias-correction mechanism, leading to faster convergence and more stable optimization.

3. Implementation:

3.1. Algorithms

Jacobian :

```
function J = computeJacobian(u, params)
   J = zeros(size(u,1), 2);
   for i = 1:size(J(:,1))  % Loop over rows (3)
        J(i, :) = [2*(params(1) - u(i, 1)) 2*(params(2) - u(i, 2))];
   end
end
```

Levenberg-Marquardt :

```
function nwparams = computeLM(params, step, J, alpha, errors)
    nwparams = params - step*pinv(J' * J + alpha*eye(size(J' * J))) * J' * errors;
end
```

Weighted Levenberg-Marquardt :

```
function nwparams = computeWLM(params, step, J, alpha, errors, W)
    nwparams = params - step*pinv(J' * W * J + alpha*eye(size(J' * J))) * J' *
W *errors;
end
```

3.2. Main Loop

The main loop of the script implementing the Levenberg-Marquardt algorithm performs the following steps:

1. Compute Residuals: The script computes the residuals as the difference between the estimated and actual radii.
2. Compute Jacobian: The script computes the Jacobian matrix, which contains the first-order partial derivatives of the residuals with respect to the parameters.
3. Compute Errors: The script computes the errors between the residuals and the Jacobian multiplied by delta parameters.
4. Update Parameters: The script updates the parameters using the optimization methods (Levenberg-Marquardt, or Weighted Levenberg-Marquardt).
5. Check Convergence: The script checks if the error falls below the specified tolerance. If it does, the loop breaks, and the optimized parameters are returned.
6. After the update of parameters, the ADAM intervenes to update the weight matrix :

```
prev_error = error_k1;

m = beta1 * m + (1 - beta1) * grad;
v = beta2 * v + (1 - beta2) * grad.^2;

m_hat = m / (1 - beta1^iter);
v_hat = v / (1 - beta2^iter);

params_k1 = params_k1 - (alpha * m_hat) ./ (sqrt(v_hat) + epsilon);

params_k0 = params_k1;
```

3.3. Parameters and Initial Conditions:

What are the initial parameters set in the script ?

```
% Algorithm parameters
lambda = 0.001;  % Damping factor
maxIter = 1000;  % Maximum number of iterations
tolerance = 1e-6;  % Convergence tolerance

% ADAM Adaptative step size
alpha = 0.01;
beta1 = 0.9;
beta2 = 0.999;
epsilon = 1e-8;
```

4. Results

4.1. Simple Levenberg-Marquardt

For this algorithm, the variance is neither introduced nor considered. The radii are fetched exactly once, as initial parameters. For this report, 3 numbers of iterations are considered : 500, 1500, 5000.

4.1.1. Iteration = 500

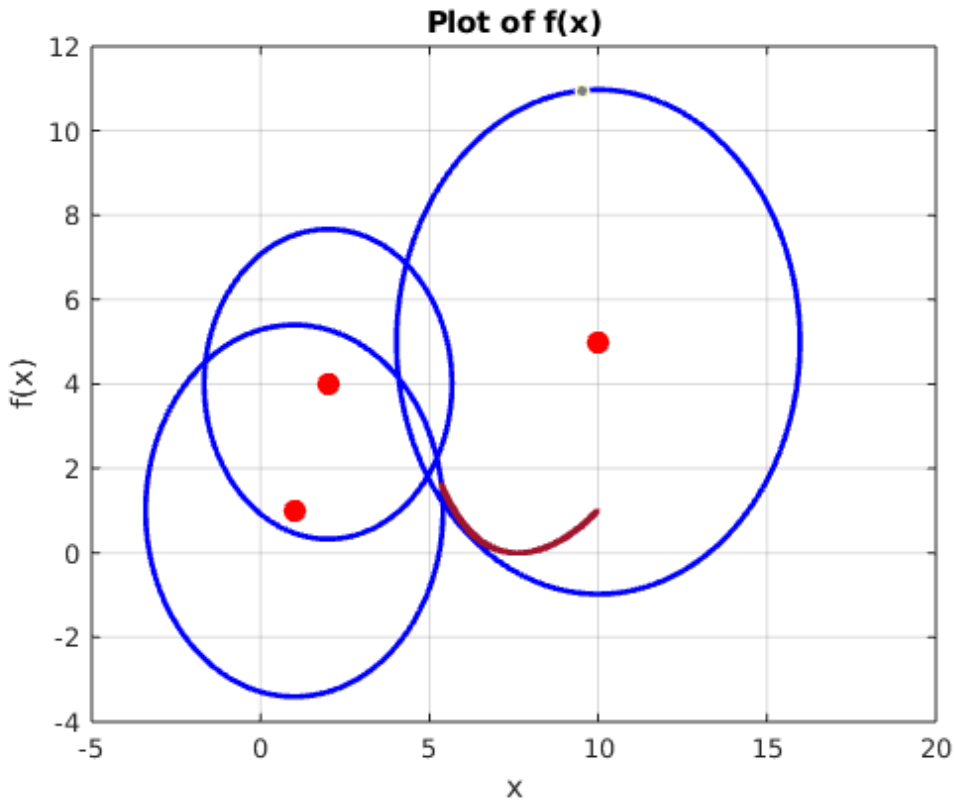


Figure 1: Output after 500 iterations

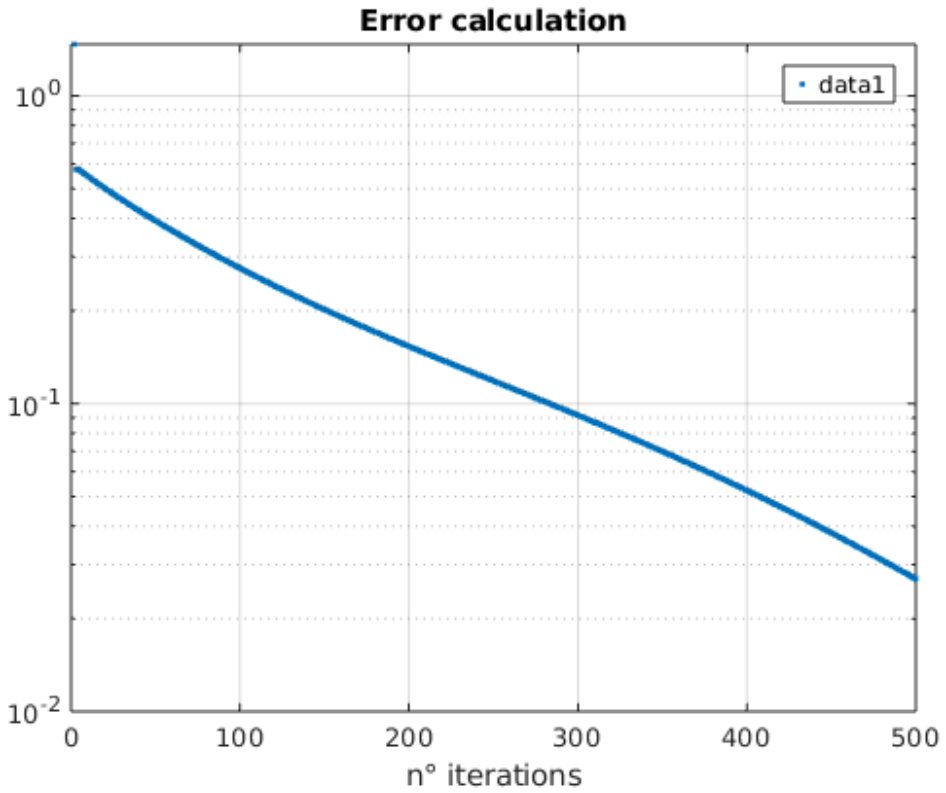


Figure 2: Errors at each iterations

The algorithm converges towards the common points, thus showing the algorithm functioning correctly. However, 500 iterations is not enough steps to converge toward the common point, as shown in the first figure.

4.1.2. Iteration = 1500

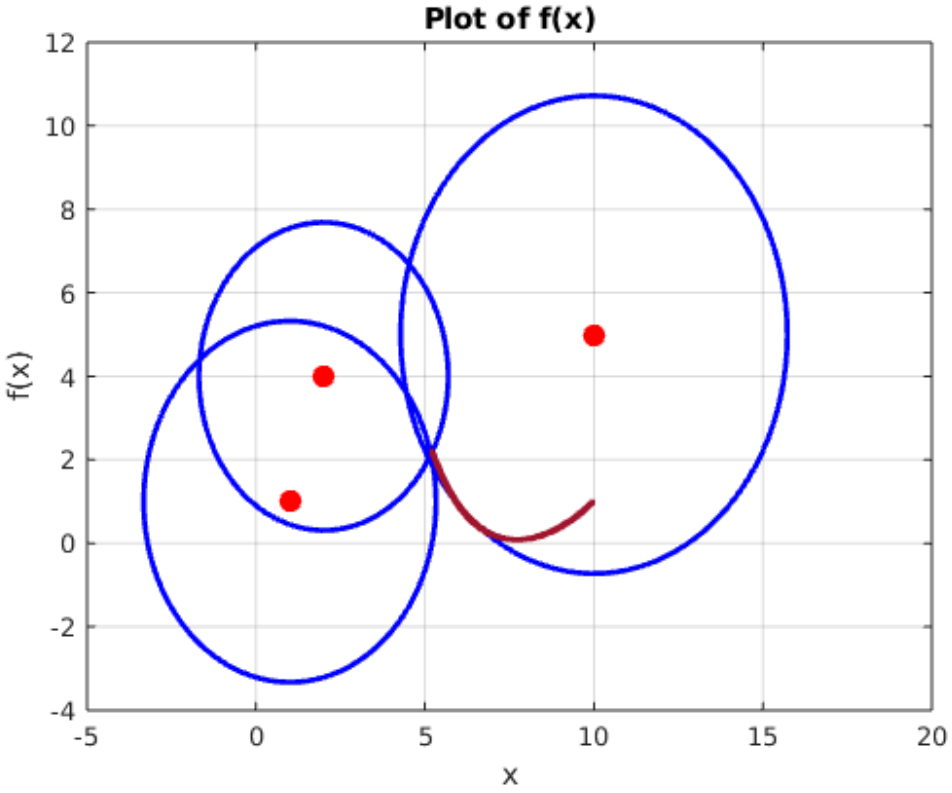


Figure 3: Output after 1500 iterations

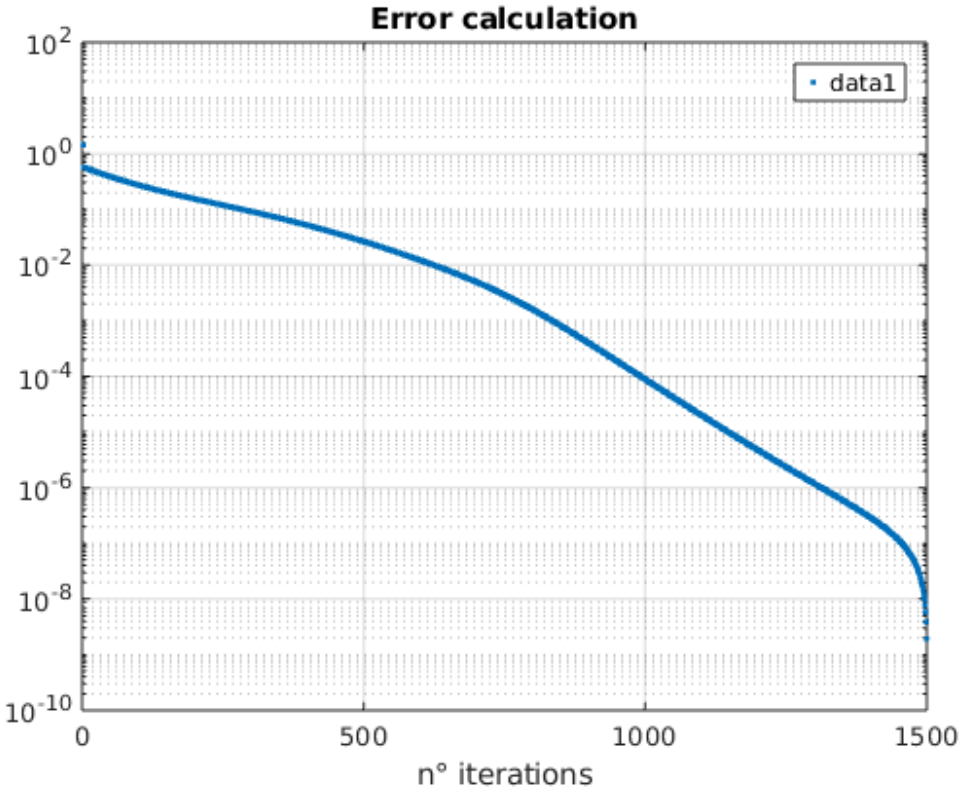


Figure 4: Errors at each iterations

The algorithm still converges correctly, albeit now with a satisfying error at the end of the algorithm, of around 10^{-8}.

4.1.3. Iteration = 5000

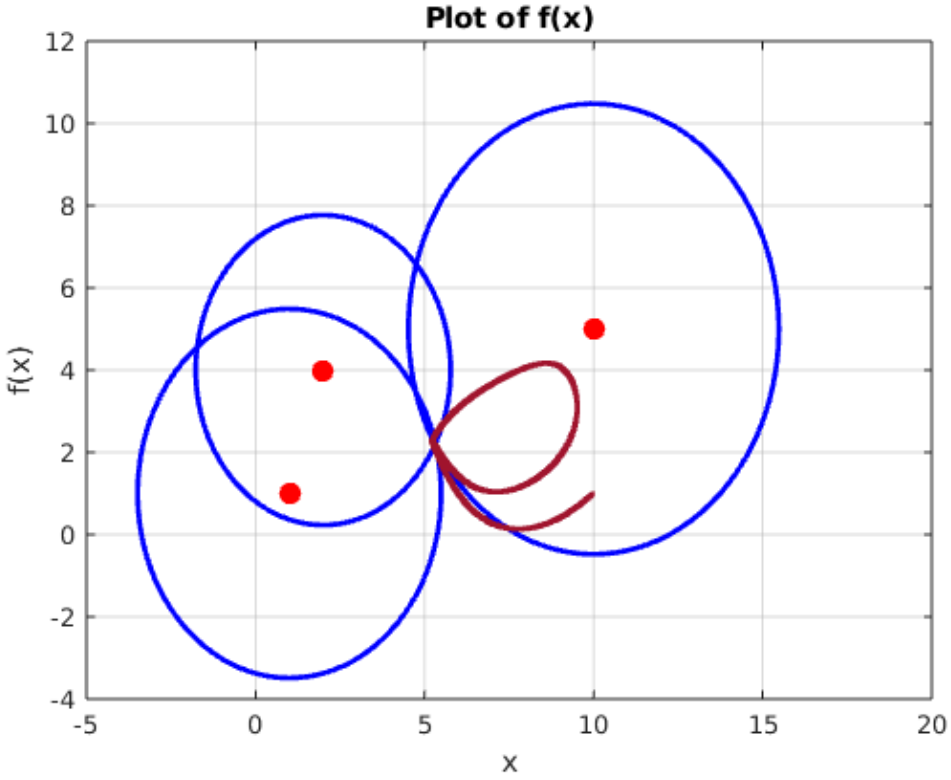


Figure 5: Output after 5000 iterations

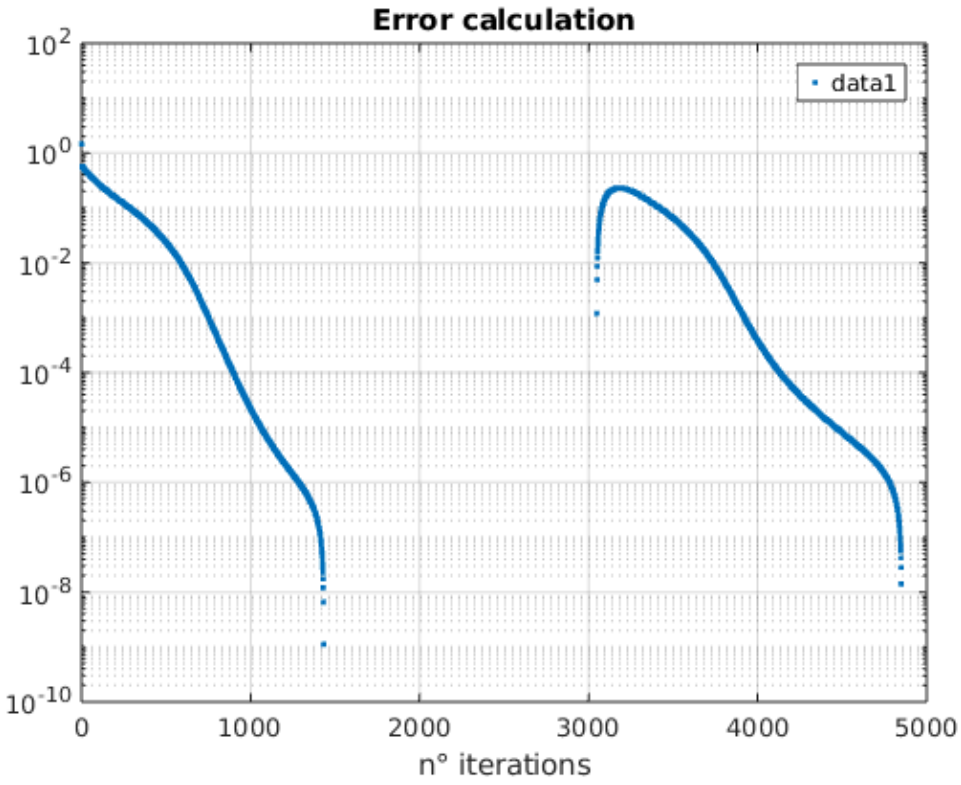


Figure 6: Errors at each iterations

Interestingly enough, the algorithm knows in this case 2 separate convergences with the same behaviours :

1. Between 0 and 1500 iterations.
2. Between 3000 and 4800 iterations.

After 1500 iterations, the gradient descent doesn't locally converge but goes instable instead, as shown by the loop. Once reaching the initial error, the gradient descent converges normally again. This example shows exactly why to stop the algorithm after reaching a satisfying level of error.

4.2. Weighted Levenberg-Marquardt

4.2.1. Initial problem

Considering the variance means radii are updated at each iteration of the main loop. This however doesn't bode well with the simple Levenberg-Marquardt algorithm who doesn't consider the variance.

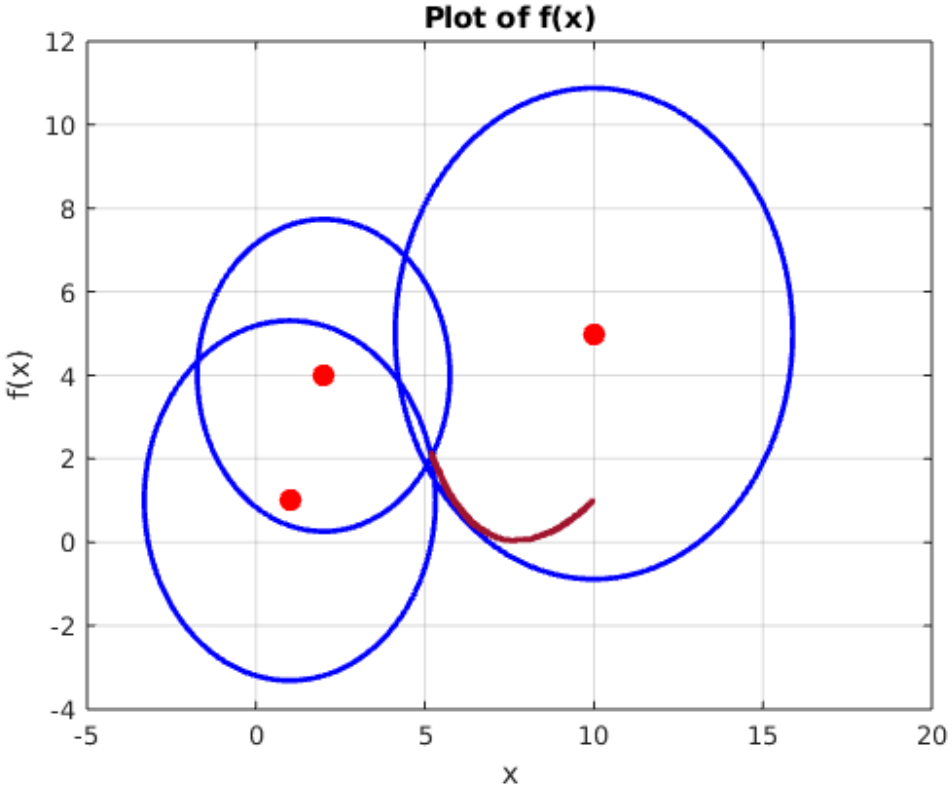


Figure 7: Output of LM algorithm with introducing variance

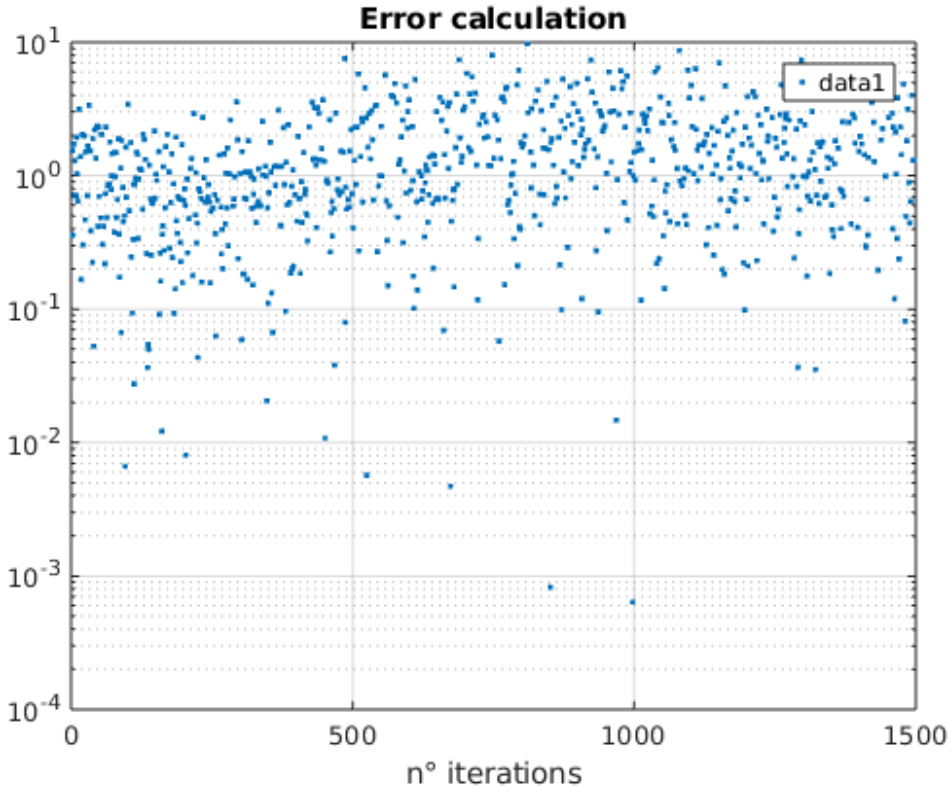


Figure 8: Computation error with a non-converging algorithm

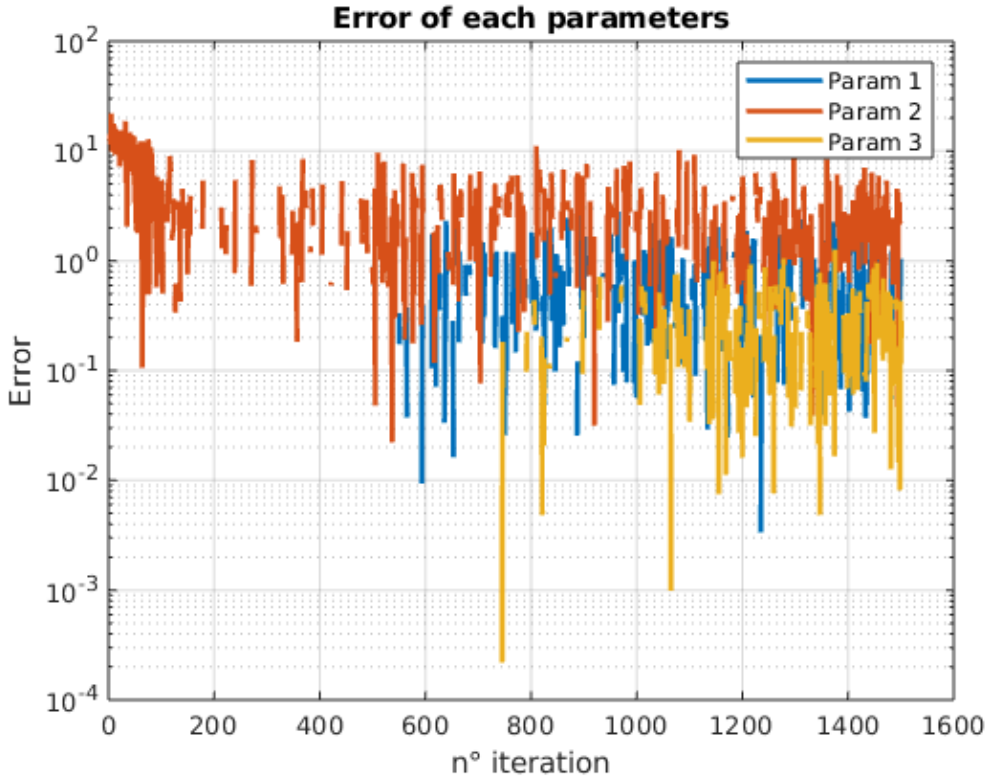


Figure 9: Errors for each parameters at every iteration

Visually, the algorithm seems to converges towards the correct common point. However, the error doesn't really go down as the algorithm progresses towards the solution, mainly because the variance is not considered and thus acts as noise in the gradient descent.

4.2.2. Introducing weighting

The weight matrix and ADAM algorithm are introduced to consider variance and thus helping converging towards the right solution. In both the error profiles, it is expected to see rapidly converging error towards far lower average for the cloud points.

4.2.3. Results

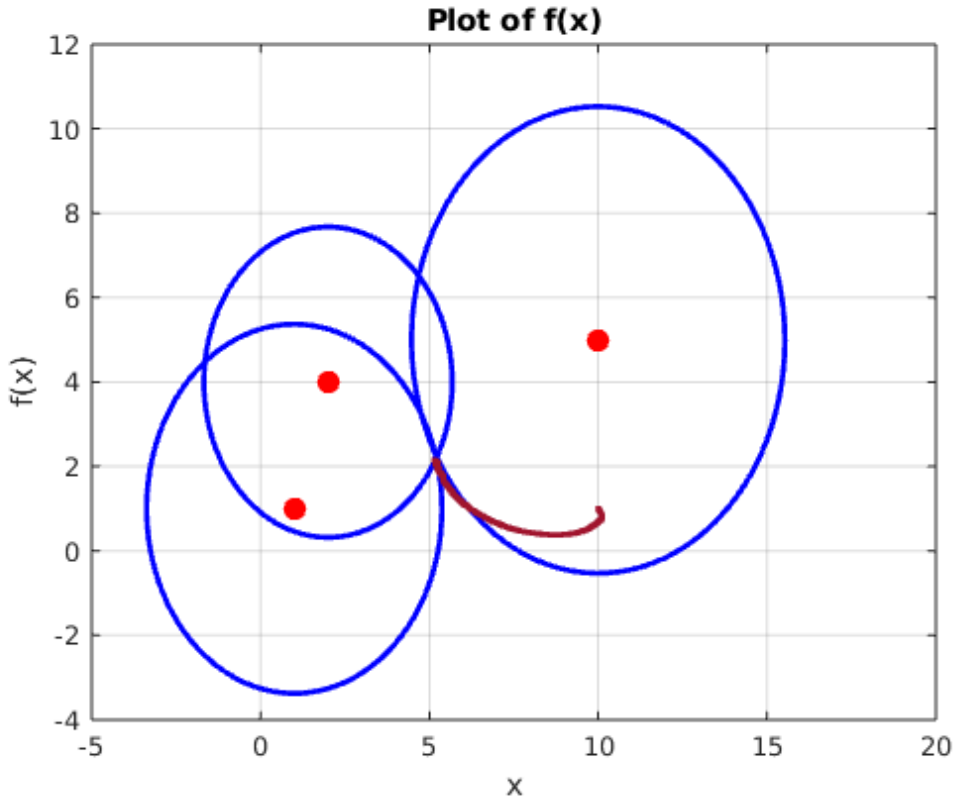


Figure 10: Output of Weighted LM algorithm with variance

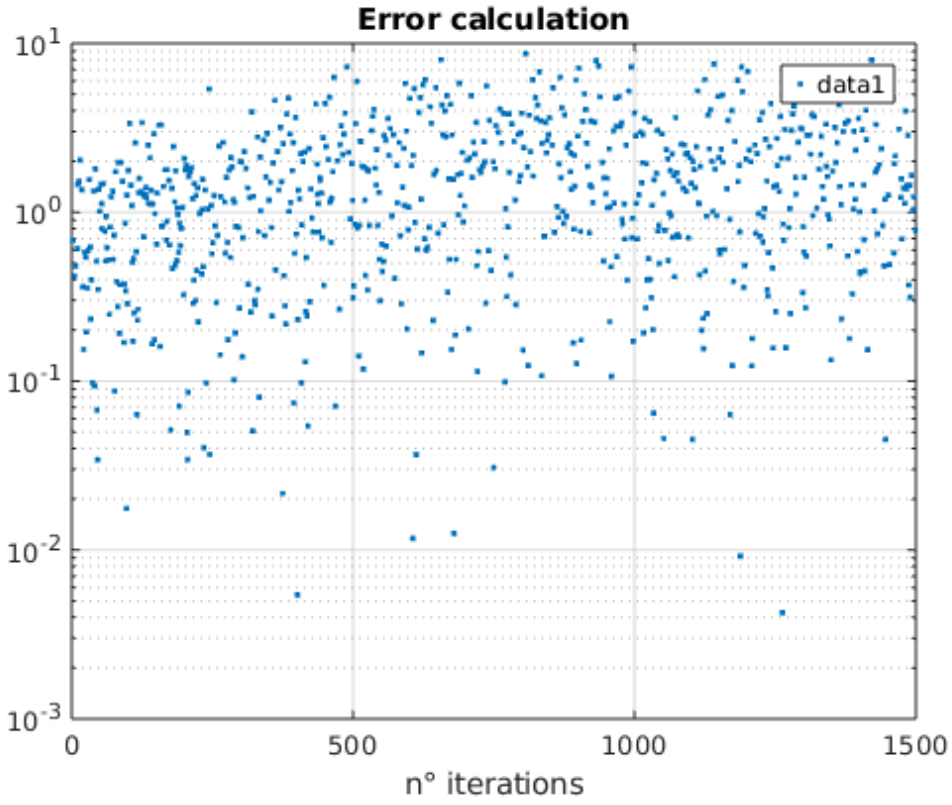


Figure 11: Computation error with a non-converging algorithm

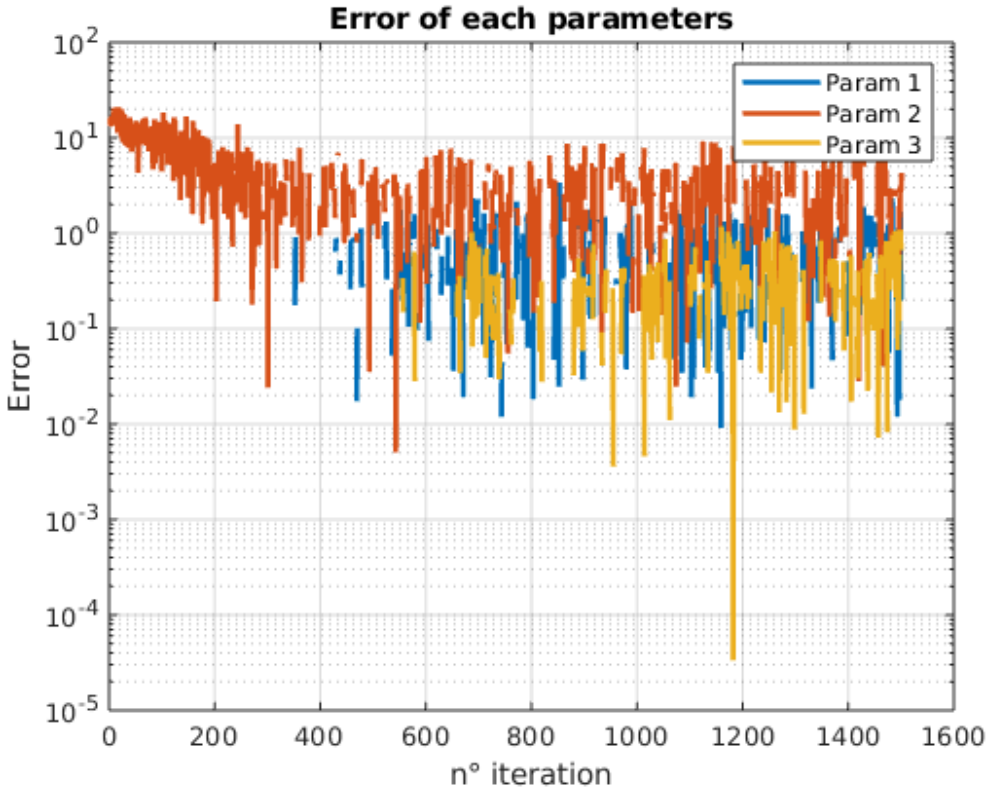


Figure 12: Errors for each parameters at every iteration

5. Conclusion

In this analysis, I explored the performance of the Levenberg-Marquardt optimization algorithm under different conditions and iterations. Initially, the algorithm was tested without considering variance, with radii fetched only once as initial parameters. I observed the algorithm's behavior over three different iteration counts: 500, 1500, and 5000.

At 500 iterations, the algorithm showed convincing signs of convergence but was insufficient to reach the correct point. By 1500 iterations, the algorithm achieved a satisfactory error level, demonstrating effective convergence. Interestingly, at 5000 iterations, the algorithm exhibited two distinct convergence phases, highlighting the importance of stopping the algorithm once a satisfactory error level is achieved to avoid instability.

When introducing variance, the simple Levenberg-Marquardt algorithm struggled, as it does not account for variance, leading to the algorithm not converging towards the optimal point. Incorporating the Weighted Levenberg-Marquardt method, who accounts for variance, has unfortunately not helped the algorithm to converge better.

Bibliography

Index of Figures

Figure 1 Output after 500 iterations 7
Figure 2 Errors at each iterations 8
Figure 3 Output after 1500 iterations 9
Figure 4 Errors at each iterations 9
Figure 5 Output after 5000 iterations 10
Figure 6 Errors at each iterations 10
Figure 7 Output of LM algorithm with introducing variance 11
Figure 8 Computation error with a non-converging algorithm 12
Figure 9 Errors for each parameters at every iteration 12
Figure 10 Output of Weighted LM algorithm with variance 13
Figure 11 Computation error with a non-converging algorithm 14
Figure 12 Errors for each parameters at every iteration 14